2026-10-30 10:55:58 **You**

how do I solve the assignment?

Write a program in Flex to
1) detect an ID of the form Start with capital letter then small letters and special characters
2) The key words of standard C program
3) TO DETECT FLOATING POINT NUMBER
4) TO DETECT POSITIVE AND NEGATIVE INTEGER

Gemini 3.8 Flash

To solve this assignment, you need to define regular expressions for each pattern and set up rule actions in Flex (Fast Lexical Analyzer).

Here is the complete solution, including the regular expression definitions, the `.l` source file, and how to compile and run it.

---

1. Pattern Breakdown & Regular Expressions

- **Identifier rule (Starts with uppercase, followed by lowercase and special characters):**
  - Starts with `[A-Z]`.
  - Followed by one or more lowercase letters or special characters (such as punctuation/symbols): `[a-z_!@#$%^&*()\-+=~ []{}|;:'",.<>?/]+`.
  - Combined regex: `^[A-Z][a-z_!@#$%^&*()\-+=~ []{}|;:'",.<>?/]+` (or simply `[A-Z][a-z_!@#$%^&*()-+=~ [\]{}|;:'",.<>?/]+` within tokens).
- **C Standard Keywords:**
  - Common standard C keywords like `int`, `float`, `char`, `double`, `if`, `else`, `while`, `for`, `return`, `void`, `switch`, `case`, `break`, `default`, etc.
  - *Important order note:* Match keywords **before** generic identifiers if there is any overlap.
- **Positive and Negative Integers:**
  - An optional or explicit sign followed by digits:
    - Positive: `\+?[0-9]+`
    - Negative: `-[0-9]+`

- **Floating-Point Numbers:**
  - An optional sign, followed by digits, a decimal point, and digits (with optional exponent notation):
    - `[+-]?[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?`
  - *Important order note:* Check for floating-point numbers **before** integers so the integer pattern doesn't prematurely consume the digits before the decimal dot.

---

2. Complete Flex Code (`scanner.l`)

Save the following code in a file named `scanner.l` :

```
</> Flex

%{
#include <stdio.h>
%}

/* Definitions / Shorthands */
DIGIT       [0-9]
FLOAT       [+-]?{DIGIT}+\.{DIGIT}+([eE][+-]?{DIGIT}+)?
POS_INT     \+?{DIGIT}+
NEG_INT     -{DIGIT}+
CUSTOM_ID   [A-Z][a-z_!@#$%^&*()\-+=~`[\]{}|;:'",.<>?/]+
WS          [ \t\r\n]+

%%

    /* 1. C Standard Keywords */
"auto"|"break"|"case"|"char"|"const"|"continue"|"default"|"do"|"double"    { printf("[C
KEYWORD]: %s\n", yytext); }
"else"|"enum"|"extern"|"float"|"for"|"goto"|"if"|"inline"|"int"|"long"     { printf("[C
KEYWORD]: %s\n", yytext); }
"register"|"restrict"|"return"|"short"|"signed"|"sizeof"|"static"|"struct"  { printf("[C
KEYWORD]: %s\n", yytext); }
"switch"|"typedef"|"union"|"unsigned"|"void"|"volatile"|"while"            { printf("[C
KEYWORD]: %s\n", yytext); }

    /* 2. Floating-Point Numbers */
{FLOAT}         { printf("[FLOATING POINT]: %s\n", yytext); }

    /* 3. Integers */
{NEG_INT}       { printf("[NEGATIVE INTEGER]: %s\n", yytext); }
{POS_INT}       { printf("[POSITIVE INTEGER]: %s\n", yytext); }

    /* 4. Custom Identifier */
```

```
{CUSTOM_ID}     { printf("[CUSTOM ID]: %s\n", yytext); }

    /* Skip Whitespace */
{WS}            { /* ignore whitespace */ }

    /* Catch-all for unrecognized characters */
.               { printf("[UNRECOGNIZED]: %s\n", yytext); }

%%

int yywrap(void) {
    return 1;
}

int main(int argc, char **argv) {
    if (argc > 1) {
        FILE *fp = fopen(argv[1], "r");
        if (!fp) {
            perror("Error opening file");
            return 1;
        }
        yyin = fp;
    }
    yylex();
    return 0;
}
```

---

3. How to Compile and Run

Run these commands in your terminal (Linux/macOS or MinGW/WSL on Windows):

1. **Generate the C file with Flex:**

   ```
   </> Bash

   flex scanner.l
   ```

   (This generates `lex.yy.c`)

2. **Compile with GCC:**

   ```
   </> Bash

   gcc lex.yy.c -o scanner
   ```

3. **Run interactively or with an input file:**

- Run with standard input (type text, press Enter, then press `Ctrl+D` to finish):

```bash
./scanner
```

- Or pass an input file:

```bash
./scanner input.txt
```

Sample Input & Output

Input (`input.txt`):

```
int x = 45;
float y = -3.1415;
int z = -90;
Abc#$! while +200
```

Output:

```
[C KEYWORD]: int
[UNRECOGNIZED]: x
[UNRECOGNIZED]: =
[POSITIVE INTEGER]: 45
[UNRECOGNIZED]: ;
[C KEYWORD]: float
[UNRECOGNIZED]: y
[UNRECOGNIZED]: =
[FLOATING POINT]: -3.1415
[UNRECOGNIZED]: ;
[C KEYWORD]: int
[UNRECOGNIZED]: z
[UNRECOGNIZED]: =
[NEGATIVE INTEGER]: -90
[UNRECOGNIZED]: ;
[CUSTOM ID]: Abc#$!
[C KEYWORD]: while
[POSITIVE INTEGER]: +200
```

2026-10-30 11:04:05 You

what is bison

Gemini 3.8 Flash

GNU Bison (commonly called **Bison**) is a general-purpose **parser generator** that is part of the GNU Project. It is the modern, backward-compatible replacement for the classic Unix tool **Yacc** (*Yet Another Compiler Compiler*).

What Does It Do?

While **Flex** acts as a **lexer** (scanning raw text and breaking it into tokens like keywords, numbers, or identifiers), **Bison** acts as a **parser** (analyzing how those tokens fit together grammatically).

- **Input:** A grammar specification file (`.y`) written using **Context-Free Grammar (CFG)** rules, typically in Backus–Naur Form (BNF).
- **Output:** C, C++, or Java source code that implements an **LALR(1)** or **GLR** parser capable of validating sentence structure and building an Abstract Syntax Tree (AST).

How Flex and Bison Work Together

In compiler design, they form the classic two-stage front end:

1. **Source Code** → **Flex (`yylex()`)**: Breaks text into individual tokens (e.g., `NUMBER` , `PLUS` , `IDENTIFIER`).
2. **Tokens** → **Bison (`yyparse()`)**: Checks token sequences against grammar rules and executes semantic actions (e.g., evaluating an expression or compiling code).

A Simple Example: A Calculator Grammar (`calc.y`)

```
</> Yacc

%{
#include <stdio.h>
int yylex(void);
void yyerror(const char *s);
%}
```

```
%token NUMBER

%%

/* Grammar rules: an expression can be addition or a simple number */
exp:
      exp '+' exp     { $$ = $1 + $3; }
    | NUMBER          { $$ = $1; }
    ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Parse error: %s\n", s);
}
```

- `exp '+' exp` defines the syntax rule.
- `{ $$ = $1 + $3; }` is the C action executed when that rule matches ($$ stores the result, $1 is the left operand,3 is the right operand).

Key Features of Bison

- **Grammar Parsing Algorithms:** Supports standard **LALR(1)**, **LR(1)**, and **GLR** (Generalized LR) parsing, which handles ambiguous or non-deterministic grammars.
- **Conflict Detection:** Identifies **shift/reduce** and **reduce/reduce** conflicts when grammar rules are ambiguous.
- **Multi-language Support:** Can generate parser implementations in **C**, **C++**, and **Java**.
- **Error Recovery:** Provides built-in hooks to catch syntax errors gracefully without crashing the parser.

2026-10-30 11:52:34 **You**

is bison used by default when I use flex commands?

Gemini 3.8 Flash

No, Bison is not used by default when you run Flex commands.

Flex and Bison are entirely separate, independent tools:

- **Flex operates standalone:** When you run `flex scanner.l`, Flex only reads your `.l` file and outputs a single C source file (`lex.yy.c`). It does not invoke Bison, require Bison, or generate parser tables.
- **Bison is optional:** You only need Bison if you are analyzing grammar or syntax trees (parsing). If your assignment only requires lexical analysis—such as tokenizing, pattern matching, counting words, or printing categories—Flex alone with a standard C `main()` function is completely self-sufficient.

You only involve Bison when you explicitly write a separate `.y` grammar file, run the `bison` command, and configure Flex to return token codes (e.g., `return TOKEN_NAME;`) instead of handling everything inside `printf` actions.

2026-10-30 12:05:13 **You**

in.txt | read_write_file.l | l2a.l | l2b.l | l2c.l | l2d.l | m.c

explain me the entire flex programming working process. I am confused about 2 types of codes:
- in one type in.txt and read_write_file.l are kept together, and run using `flex read_write_file.l` and the follow lex.yy.c commands.
- in another type l2a.l, l2b.l, l2c.l, l2d.l are run alongside m.c using the same flex commands. But I am confused about the lex.yy.c generation during the flex command usage. does the lex.yy.c generated for l2a.l and l2b.l the same?

Gemini 3.8 Flash

How Flex Works Under the Hood

Flex is not a C compiler; it is a **code generator**. Its job is to take a set of regular expressions and actions described in a `.l` file, generate a Deterministic Finite Automaton (DFA) state machine in pure C, and output that state machine into a standard C source file—by default named `lex.yy.c`.

```
</> Plain text

[your_file.l] ---> (flex) ---> [lex.yy.c] ---> (gcc) ---> [a.out / executable]
                                    ^
```

```
                    |
            Takes input from file/stdin
```

1. **`flex file.l`** : Reads `file.l` and translates your pattern rules into a function named `yylex()` . It writes everything into `lex.yy.c` .
2. **`gcc lex.yy.c -o my_scanner`** : Standard GCC compilation that turns the generated C code into an executable.
3. **Execution**: When `yylex()` runs, it reads characters from the file pointer `yyin` (which defaults to standard input `stdin` unless redirected).

Does `lex.yy.c` generated for `l2a.l` and `l2b.l` stay the same?

No, they are completely different, and running `flex` overwrites the file.

Whenever you run:

```
</> Bash

flex l2a.l
```

Flex generates `lex.yy.c` containing the logic and patterns specific to `l2a.l` .

If you immediately run:

```
</> Bash

flex l2b.l
```

Flex **completely overwrites** `lex.yy.c` with the code corresponding to `l2b.l` .

If you compile and run between those steps, you are running two distinct programs:

```
</> Bash

# Step 1: Build and run l2a
flex l2a.l                # creates lex.yy.c containing l2a's rules
gcc lex.yy.c -o exec_a    # compiles exec_a
./exec_a

# Step 2: Build and run l2b
flex l2b.l                # OVERWRITES lex.yy.c with l2b's rules
gcc lex.yy.c -o exec_b    # compiles exec_b
./exec_b
```

(Note: If you ever need custom output names directly from Flex, you can use the `-o` flag: `flex -o lex.l2a.c l2a.l`).

Dissecting the Two Scenarios

Scenario 1: `read_write_file.l` with `in.txt`

Look at how `main()` is set up in `read_write_file.l` :

```c
int main()
{
    yyin = freopen("in.txt", "r", stdin);      // Redirects scanner input to in.txt
    yyout = freopen("out.txt", "w", stdout);   // Redirects scanner output to out.txt
    yylex();
    fclose(yyin);
    fclose(yyout);
    return 0;
}
```

- **Role of `in.txt` :** It is simply raw data. It is **not** compiled.
- **Role of `read_write_file.l` :** It is the Flex specification.
- **Execution:**
  1. `flex read_write_file.l` → creates `lex.yy.c` .
  2. `gcc lex.yy.c -o rw_prog` → creates the executable `rw_prog` .
  3. `./rw_prog` → The program reads characters from `in.txt` , matches rules like `LB|RB|LP|RP` or `.+` , and prints the results directly into `out.txt` .

Scenario 2: `l2a.l` , `l2b.l` , `l2c.l` , `l2d.l` with `m.c`

Here, `m.c` is **not source code to be compiled by GCC**. It is used as an **input text file** containing pseudo-C tokens (like `hashontorvukto` , `integer` , `colon` , `assign` , etc.).

Each `.l` file (`l2a.l` , `l2b.l` , `l2c.l` , `l2d.l`) is an independent exercise that opens `m.c` inside its own `main()` function using:

```c
yyin = fopen("m.c", "r");
```

Each `.l` file performs a different task on the contents of `m.c` :

- `l2a.l` **(Echo non-empty lines):**
  - Pattern: `.+` prints every non-empty line as-is.
- `l2b.l` **(Default pass-through):**
  - Pattern: Empty rules section. By default, Flex echoes every character it does not match directly to output.
- `l2c.l` **(Line & Character Counter):**
  - Increments `num_lines` on `\n` and `num_chars` on any character (`.`), then prints the counts in `main()` .
- `l2d.l` **(Number Tokenizer):**
  - Recognizes digit sequences (`{DIGIT}+`) and floating point numbers (`{DIGIT}+"."{DIGIT}*`), ignoring any other character (`.`).

Comparison Summary

Aspect	`read_write_file.l` Setup	`l2a.l` ... `l2d.l` Setup
Input File	`in.txt` (via `freopen` on `stdin`)	`m.c` (via `fopen` directly into `yyin`)
Output Target	Redirected to `out.txt`	Printed to console (`stdout`)
Role of `.c` File	None (only compiled `lex.yy.c`)	`m.c` acts strictly as test data, not as C code
`lex.yy.c` Behavior	Generated from `read_write_file.l`	Generated separately for whichever `.l` file you feed to `flex`

2026-10-30 12:06:26 **You**

do we define regular expressions in .l file? what is the purpose of doing so? was flex build before .c or afterward?

Gemini 3.8 Flash

1. Do we define regular expressions in a `.l` file?

Yes, absolutely. Regular expressions (regex) are the core foundation of a `.l` (Lex/Flex) file.

In a `.l` file, regular expressions appear in two main places:

1. **Definitions Section (Top):** Giving shorthand names to regular expressions so they can be reused:

```
</> Flex

DIGIT    [0-9]
LETTER   [a-zA-Z]
```

2. **Rules Section (Between `%%` and `%%`):** Pairing regular expressions with actions written in C:

```
</> Flex

%%
{DIGIT}+      { printf("Found an integer: %s\n", yytext); }
{LETTER}+     { printf("Found a word: %s\n", yytext); }
%%
```

---

2. What is the purpose of doing so?

The purpose of defining regular expressions in a `.l` file is **pattern-driven lexical analysis (tokenization)**:

- **Automating State Machine Creation:** Writing a lexical scanner by hand in pure C requires designing complex finite state machines, nested `switch` / `if-else` blocks, buffer tracking, and lookahead logic. By defining high-level regular expressions, you let Flex compile those patterns automatically into an optimized **Deterministic Finite Automaton (DFA)** table in C.
- **Separation of Concerns:** It isolates *what* pattern to look for (the regex) from *what to do* when found (the C action code).
- **Efficiency:** Flex implements fast table-driven scanning engines that process input text character by character in linear time ($O(N)$), which is faster and far less error-prone than manually coded string parsing.

---

3. Was Flex built before `.c` or afterward?

Flex was built long afterward.

Here is the chronological timeline:

- **The C Programming Language (`.c`):** Developed by Dennis Ritchie between **1969 and 1972** at Bell Labs, with standard adoption solidifying through the 1970s and 1980s.
- **Lex (The Original):** Written by Mike Lesk and Eric Schmidt in **1975** at Bell Labs to generate C code for the Unix operating system.
- **Flex (Fast Lexical Analyzer):** Written by Vern Paxson around **1987** as a faster, modern, open-source rewrite of AT&T's original Unix `lex` .

Because C already existed, Flex was **written in C** specifically to **generate standard C code (`lex.yy.c`)** as its output.